Until the middle of the 1990s, just before the internet revolution, it was usual to build applications that only ever ran on a single computer (a single CPU). If an application wasn't fast enough, the standard response was to wait for faster CPUs to be developed; there was no need to change the code. Problem solved. Programmers worldwide got a free lunch, and life was good.

NOTE In 2005, Herb Sutter wrote in *Dr. Dobb's Journal* about the need for a fundamental change (http://www.gotw.ca/publications/concurrency-ddj.htm): he said that the limit of increasing CPU clock speeds had been reached, and the free lunch was over.

Around 2005, when Herb Sutter wrote his excellent article, companies ran applications on clustered multiprocessor servers (often no more than two or three—the spare was in case one of them crashed). Support for concurrency in programming languages was available but limited and was considered black magic by many mere mortal programmers. Sutter predicted that "programming languages . . . will increasingly be forced to deal well with concurrency."

Let's see what has changed since then. Fast-forward to today, and applications run on large numbers of servers in the cloud; many systems are integrated across many data centers. The ever-increasing demands of end users push the performance and stability requirements for the systems you build. Concurrency is prevalent, and it is here to stay.

But support for concurrency in most programming languages has hardly changed, especially on a JVM. Although the implementation details of concurrency APIs have improved, you still have to work with low-level constructs like threads and locks or fibers. Threads are